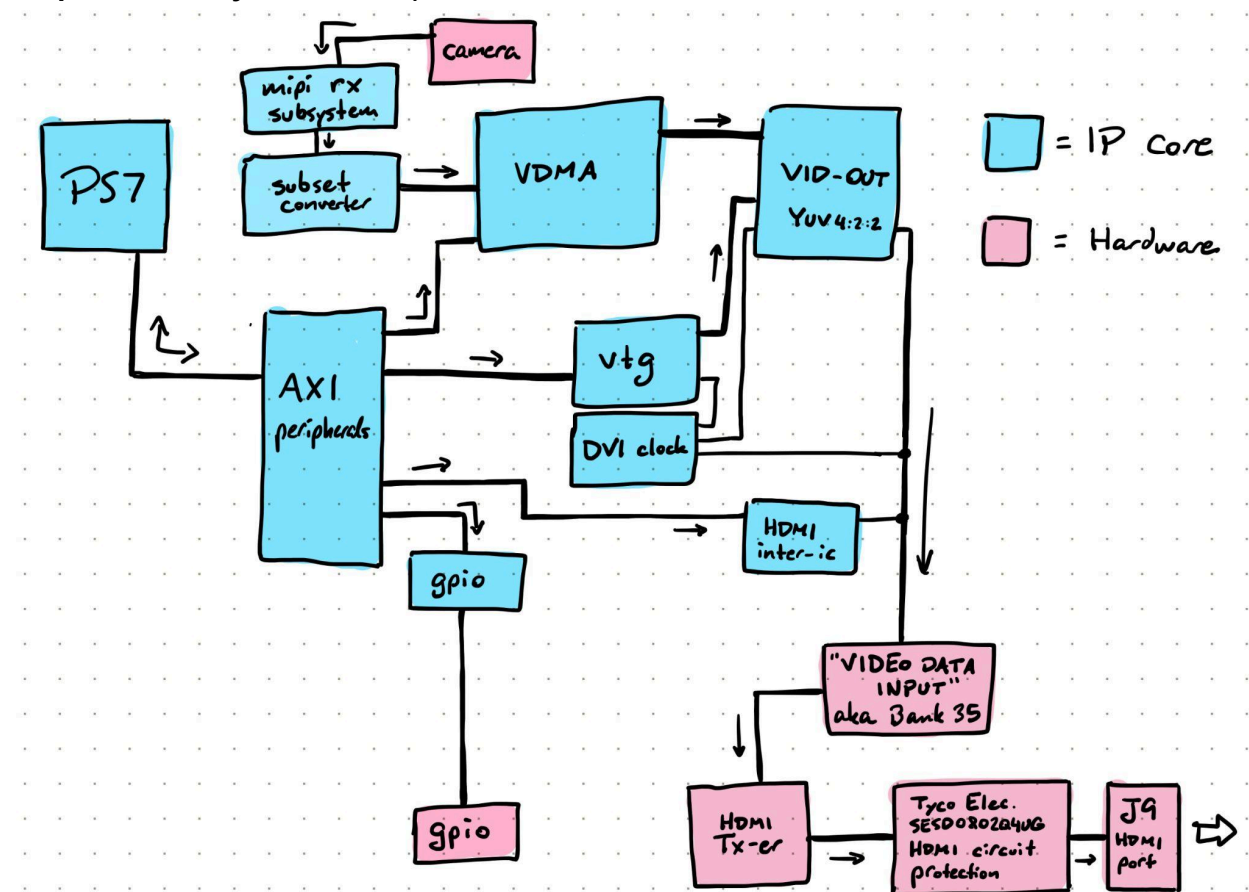


Contributions:

- (A) Caleb Hemmestad - 25%
- (A) Wilson Diep - 25%
- (B) Penny Mayer - 25%
- (B) Sandee Madan - 25%

In your writeup provide the following:

- A detailed system diagram that illustrates the interconnection between the various modules in the system, both at the IP core level (i.e. the components in your VIVADO design) as well as the board level (i.e. the various chips that work together to connect the output video to your monitor)



A detailed description of how the hardware in the starter MP-2 design is intended to operate.

The camera feeds data into the `mipi_cs2_rx_subsystem` IP. This converts the sensor data into a stream of raw pixel values, and sends it to a subset converter that converts each pixel from 10-bit to 8-bit data. The 8-bit pixel stream goes into the VDMA and is stored as a framebuffer. The framebuffer is streamed from the VDMA to the `axi4s_vid_out` IP, which combines with timing data from the Video Timing Generator and DVI clock IPs to generate an hdmi signal. This is passed off to the ZedBoard's hardware, which has an ASIC for transmitting the stream out of fpga-land, and a protection chip made by Tyco electronics.

Describe in your writeup what changes you made, and save a copy of any files modified (presumably only camera_app.cc) during this process into a folder named part3/

We modified the background pattern id register (0x20) and set it to be 0x13 to mimic the HDMI color test output

1) Note that several of these ports in the XDC file are paired together, with one port ending in _p and the other ending in _n. In your writeup, briefly describe what this pairing of signals signifies, and what this configuration is typically used for.

The same signal is sent through with one being the positive and one being the negative of the original signal. This configuration is typically used for reducing noise.

2) We modified the 16-bit “mipi_csi2_rx” IP core output by keeping bits 9:2, and appending the 8-bit value “10000000” (see step 2.i), before passing this signal to the VDMA. Explain why this is an appropriate value to append, and why appending “00000000” would not make sense.

The camera sensor is producing 10-bit values. We discard the least significant two bits so that we are dealing with an 8-bit value, for convenience at the cost of a little dynamic range.

The VDMA is expecting 16-bit values, where the least significant 8 bits are luminance and the most significant 8 bits are chroma, for either red or blue channels.

If we want our grayscale camera pixel value to be the luminosity, we need a non-zero chroma value to prevent the rendered image from being green. To keep things grayscale, a consistent chroma value of 10000000 on all pixels is sufficient.

In your writeup, briefly explain why the camera at this stage is not outputting any color.

As explained above, we are using the camera's output as the luminosity value in our color signal. Chroma data is hardcoded and constant, so there is no color.

Describe in your writeup what changes you made, and save a copy of any files modified (presumably only camera_app.cc) during this process into a folder named part5/

I used two nested for loops, going to 1920 and 1080, and incremented both by two per iteration. The inner loop considers the 2x2 area of pixels representing one BGGR bayer square. No averaging of multiple pixels is done when determining the color data, but the results are visually acceptable. The BGGR values are used to calculate Y, and then Cb and Cr are calculated from Y and the RGB values. Y, Cb, and Cr are then packed into 16-bit values and written to the VDMA's output buffer.

Look at the VDMA initialization code in function fmc_imageon_enable(), and infer from the Red, Green, and Blue examples how the 16-bit 4:2:2 YCbCr format is encoded. Briefly describe this in your writeup, and use this format as the output of your camera_loop() conversion pass.

The general format is (binary) CCCC CCCC YYYY YYYY.

C represents chroma, and each 16-bit value alternates between Cb and Cr. The zero-point for chroma is 128.

Y represents luminance. Luminance goes from 0-255, with no zero-point adjustment.

In your writeup, describe the performance of your software-based color conversion (in terms of frames per second), and how you measured it.

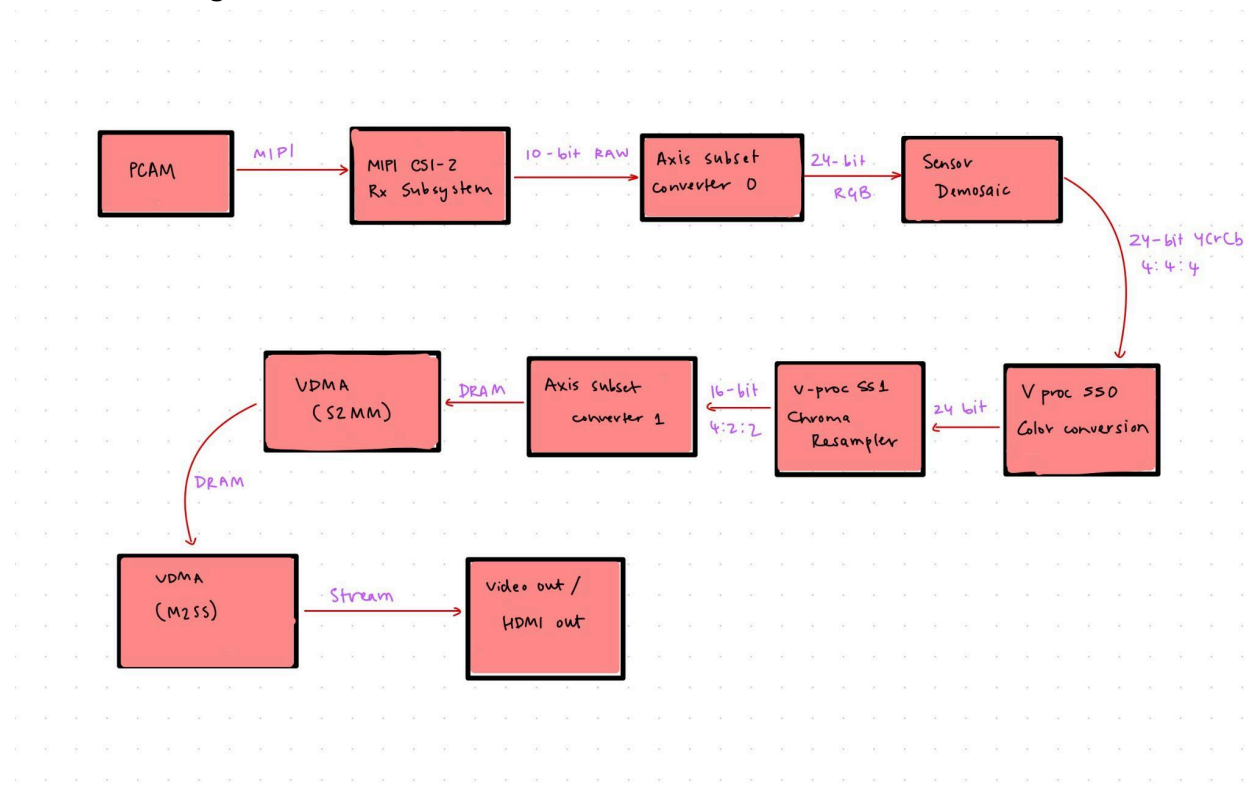
Timing was measured by the built-in XTime_GetTime function with XTime variables. FPS comes from $(\text{endTime} - \text{startTime}) / (\text{number of frames})$.

The initial version of the software-based color conversion ran at 6 fps. With optimization, it rose to 16 fps, and then 42 fps (!). The biggest optimization came from converting the floating-point multiplications to power-of-two divisions (aka 1 to 3 rightward bit shifts). I planned to use this project as an excuse to learn inline assembly, but, in the end it wasn't needed, and I'd rather be home for supper.

in your writeup, describe your testing methodology

I used a pair of pens, one blue and one red, to test for backwards chroma. I also wear a colorful, rainbow winter coat, which made a good subject for test photos while we dialed in the color algorithm.

The image pipeline should proceed from Pcam -> MIPI CSI-2 Rx Subsystem -> axis_subset_converter (8-bit output) -> (8-bit input) v_demosaic_0 -> v_proc_ss0 (Color Conversion Only) -> v_proc_ss1 (422-444 Chroma Resampling Only) -> axis_subset_converter -> vdma_S2MM -> vdma_MM2S -> vid_out -> hdmi_out. Provide a diagram for this awesome pipeline in your writeup, making sure to label the bit width of the relevant signals.



In your writeup, describe the performance of your image processing pipeline (in terms of frames per second), and how you measured it.

We are getting around 24-25fps with the hardware pipeline. It is measured by watching the parkptr and seeing how many buffer switches there have been in 1000ms.